

OpenMP Tasks



Escola Regional de Alto Desempenho

Região Sudeste

Juiz de Fora, MG - 12 a 14 de agosto de 2026

Conteúdo do curso (primeira parte)

- Exposição teórica
 - Introdução ao paralelismo com `parallel`, `single` e `task`.
 - Diferença entre tarefas implícitas e explícitas.
 - Ordem de execução não determinística.
 - Sincronização básica com `taskwait`.
 - Escopo de variáveis em tarefas: `shared`, `firstprivate` e cuidados com variáveis locais.
 - Granularidade de tarefas e uso da cláusula `if` para reduzir overhead.
- Prática em laboratório

Conteúdo do curso (segunda parte)

- Exposição teórica
 - Tasks em estruturas irregulares, como listas encadeadas.
 - Dependências entre tarefas com `depend(in)`, `depend(out)` e `depend(inout)`.
 - Diferença entre `barrier`, `taskwait` e `taskgroup`.
 - Ponto de escalonamento com `taskyield`.
 - Paralelismo de laços com `taskloop`, incluindo `grainsize`, `numtasks`, `nogroup` e `taskloop simd`.
- Prática em laboratório

Referências



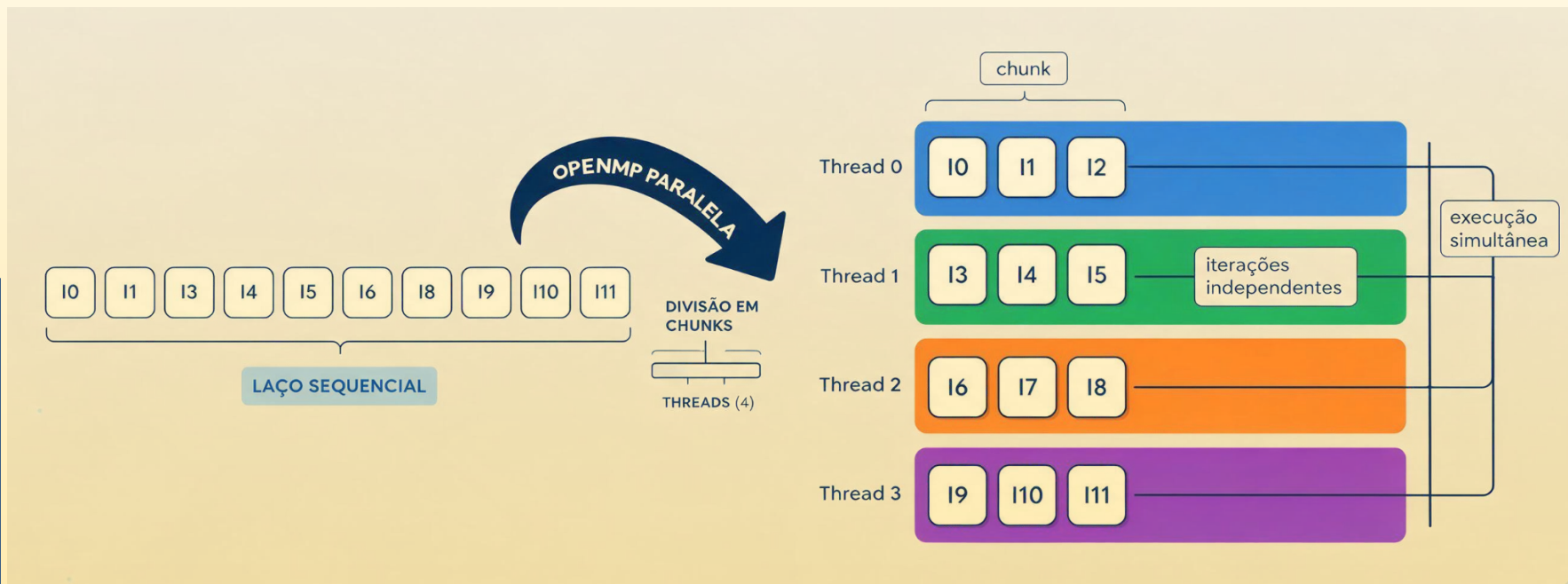
<https://programacao-paralela-e-distribuida.github.io>

Primeira Parte

Paralelismo de laços

- Fundamental no OpenMP desde as primeiras versões.
- Muito usado para acelerar laços computacionalmente intensivos.
- Baseia-se na divisão das iterações em blocos (*chunks*).
- Cada bloco é distribuído para diferentes *threads*.
- Requisito essencial: não pode haver dependências entre iterações.
- Ou seja, o resultado de uma iteração não deve depender de outra.

Paralelismo de laços



Paralelismo de tarefas

- Em problemas com execução irregular, o paralelismo de laços pode ser menos eficiente.
- O paralelismo de tarefas (*tasks*) oferece uma abordagem mais flexível.
- Ele permite executar concorrentemente diferentes tarefas independentes no mesmo programa.
- É útil para estruturas não repetitivas e com melhor granularidade de paralelismo.
- Também é aplicada em *offloading* de *kernels* computacionalmente intensivos e em computação irregular para aceleradores.

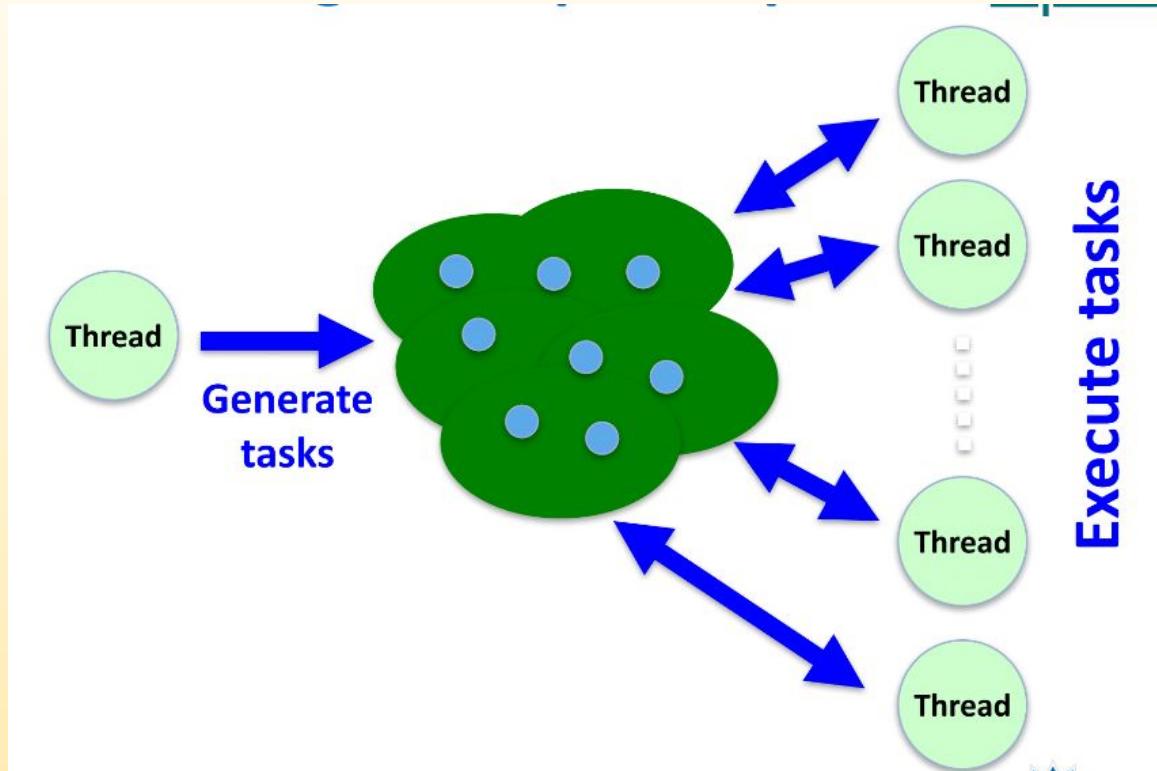
Tarefas

- As tarefas são unidades independentes de trabalho compostas por:
 - código a ser executado
 - ambiente de dados (que é iniciado na sua criação)
 - variáveis de controle internas (IVC)
- As *threads* executam o trabalho definido por cada tarefa.
- É responsabilidade do programador garantir que tarefas diferentes possam ser executadas simultaneamente, sem conflito no acesso aos dados compartilhados.
- O ambiente de execução (*runtime system*) decide se as tarefas serão adiadas ou executadas imediatamente.

Criação de tarefas

- Se a execução da tarefa for adiada, a tarefa será colocada em um *pool* de tarefas.
- A *thread* que executa uma tarefa pode diferir da *thread* que a criou originalmente.
- As tarefas são criadas:
 - ao alcançar uma região paralela → tarefas **implícitas** são criadas (uma por *thread*);
 - ao encontrar uma construção de **task** → uma tarefa **explícita** é criada;
 - ao encontrar uma construção de **taskloop** → tarefas explícitas para cada bloco (*chunk*) são criadas;
 - ao encontrar uma construção **target** → uma tarefa **target** é criada.

Criação de tarefas



```
#pragma omp task [clause]
{
    ...
}
```

Classificação das tarefas

- **Tarefa não vinculada:** pode ser retomada por qualquer *thread* da equipe quando sua execução é suspensa.
- **Tarefa não postergada:** sua execução não é adiada, a região geradora fica suspensa até a tarefa terminar.
- **Tarefa incluída:** é executada imediatamente pela *thread* que a encontra, de forma sequencial.
- **Tarefa mesclada:** compartilha o mesmo ambiente de dados da região geradora, incluindo as ICVs.
- **Tarefa final:** faz com que todas as suas tarefas filhas se tornem tarefas finais e incluídas.

Escalonamento de tarefas

- A execução da região tarefa atual pode ser suspensa, para ser depois retomada, em um **ponto de escalonamento** de tarefa.
- O ponto de conclusão da tarefa, quando a *thread* em execução pode mudar para uma região de tarefa diferente, também define um ponto de escalonamento de tarefa.
- Os pontos de escalonamento de tarefas dividem dinamicamente as regiões de tarefas em partes, onde cada parte é executada na ordem em que são encontradas e sem interrupções do início ao fim.

Escalonamento de tarefas

- OpenMP define os seguintes pontos para o escalonamento de tarefas:
 - No ponto de encontro de uma construção `task`;
 - No ponto de encontro de uma construção `taskwait`;
 - No ponto de encontro de uma construção `taskyield`;
 - Ao final de uma região `taskgroup`;
 - No ponto de encontro de uma barreira (`barrier`) implícita or explícita;
 - No ponto imediatamente após a geração de uma região `target`;
 - No ponto de conclusão de uma construção `task`.
- Todas as tarefas explícitas geradas dentro de uma região paralela têm a garantia de serem completadas na saída da próxima barreira implícita ou explícita dentro da região paralela.

Pontos de sincronização

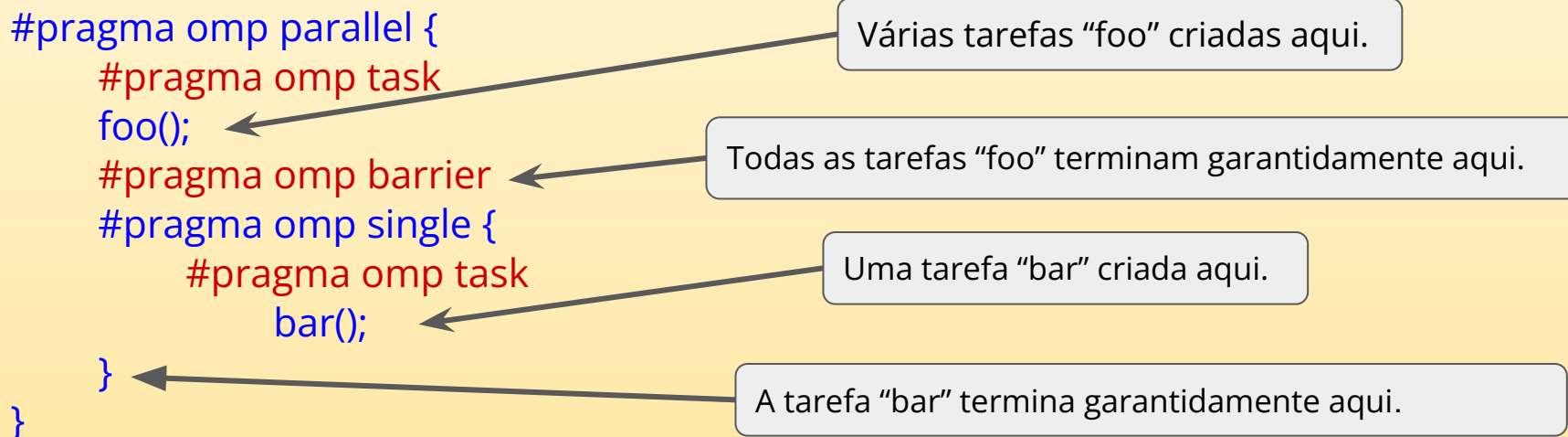
- Em certos pontos do código, que são chamados de **pontos de sincronização** de tarefas, é garantido que todas as tarefas anteriores já terminaram.
- Alguns dos pontos **explícitos** de sincronização de tarefas:
 - `#pragma omp taskwait`
 - `#pragma omp barrier`
 - `#pragma omp taskgroup`
- Os pontos **implícitos** de sincronização ocorrem, por exemplo, em todas as barreiras implícitas do OpenMP, como as que ocorrem ao final de uma região paralela ou de uma diretiva **single**.

Diretiva task

- Quando uma *thread* encontra uma construção **task**, uma tarefa é gerada a partir do código associado ao bloco estruturado correspondente.
- Por bloco estruturado entenda-se um trecho de código delimitado por chaves { ~ } que define uma unidade lógica e coesa de operações, com uma única entrada no topo e uma única saída na parte inferior.
- Um bloco estruturado não pode ser alcançado a partir de um desvio e o ponto de saída não pode ser um desvio para fora do bloco estruturado.

OpenMP tasks

- Construção de tarefa (task construct) – Uma diretiva task mais um bloco estruturado.
- Região de tarefa (task region) – um sequência dinâmica de instruções produzidas pela execução de uma tarefa por uma thread.



Exemplo #1

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc char * argv[]) {
    #pragma omp parallel {
        printf ("A ");
        printf ("race ");
        printf ("car ");
    } // Fim da Região Paralela
    printf ("\n");
    return(0);
}
```

As threads são executadas em paralelo.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A race car A race car
$ ./a.out
A race A car race car
```

Exemplo #2

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc char * argv[]) {

#pragma omp parallel {
#pragma omp single {
    printf ("A ");
    #pragma omp task
    printf ("race ");
    #pragma omp task
    printf ("car ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas podem ser executadas em qualquer ordem.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A race car
$ ./a.out
A race car
$ ./a.out
A car race
```

Exemplo #3

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc char * argv[]) {

#pragma omp parallel {
#pragma omp single {
    printf ("A ");
    #pragma omp task
    printf ("race ");
    #pragma omp task
    printf ("car ");
    printf("is fun to watch ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas são executadas no ponto de execução das tasks.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A is fun to watch race car
$ ./a.out
A is fun to watch race car
$ ./a.out
A is fun to watch car race
```

Exemplo #4

```
#include <stdlib.h>
#include <stdio.h>

int main( int argc char * argv[]) {

#pragma omp parallel {
#pragma omp single {
    printf ("A ");
    #pragma omp task
    printf ("race ");
    #pragma omp task
    printf ("car ");
    #pragma omp taskwait
    printf("is fun to watch ");
} // Fim da Região Paralela

printf ("\n");
return(0);
}
```

As tarefas são executadas primeiro agora.

```
$ gcc -fopenmp hello.c
$ export OMP_NUM_THREADS=2
$ ./a.out
A car race is fun to watch
$ ./a.out
A car race is fun to watch
$ ./a.out
A race car is fun to watch
```

Cláusulas da diretiva task

- `#pragma omp task [cláusula[,cláusula]*]`

- `if (expression)`
- `final (expression)` - Quando a expressão final é avaliada como verdadeira, a tarefa não terá descendentes.
- `mergeable` - Uma tarefa mesclada é uma tarefa cujo ambiente de dados é o mesmo que a região da tarefa geradora.

Estratégia de encerramento

- `untied` - Se a tarefa estiver vinculada, é garantido que uma mesma thread executa todas as partes da tarefa, mesmo que a execução da tarefa tenha sido temporariamente suspensa.
- `priority (value)` - Um valor numérico não negativo, que recomenda que uma tarefa com alta prioridade seja executada antes de uma tarefa com menor prioridade.

Escalonamento das tasks

- `default (shared | firstprivate | none)`
- `private (list)`
- `firstprivate (list)`
- `shared (list)`

Escopo das variáveis

- `depend (list)` - Veja exemplo

Sincronização

Cláusulas if e final

- As cláusulas **if** e **final** são usadas para controlar a geração e a execução das tarefas, com pequenas diferenças entre elas.
- A cláusula **if** recebe uma expressão booleana que determina se a tarefa deve ser gerada ou não.
- Se a expressão for **falsa**, uma **tarefa não postergável** é gerada e a *thread* que a encontra deve executá-la sequencialmente até o final.
- Se a expressão da cláusula **final** for avaliada como **verdadeira**, a tarefa gerada será uma **tarefa final e incluída**.
- Neste caso, a tarefa e todas as suas descendentes serão executadas sequencialmente pela *thread* que a encontrou, sem serem colocadas no *pool*.

Cláusula if e final

```
void trabalho(int n) {  
    if (n <= 1) return;  
    #pragma omp task if(n > 4) final(n <= 4)  
        printf("Task executando com n = %d na thread %d\n", n,  
omp_get_thread_num());  
        trabalho(n - 1);  
}  
int main() {  
    #pragma omp parallel  
        #pragma omp single  
            trabalho(8);  
    return 0;  
}
```


Cláusula mergeable

- Quando a cláusula **mergeable** está presente, a tarefa gerada é **mesclável**, compartilhando o mesmo ambiente de dados e variáveis de controle internas da região geradora.
- Isso permite ao compilador unir duas ou mais tarefas em uma única tarefa quando executadas na mesma *thread*, melhorando a eficiência e reduzindo a sobrecarga de tarefas pequenas.
- Deve-se observar com cuidado o escopo das variáveis, de modo a garantir uma execução correta do programa, caso o compilador decida-se pela união das tarefas.

Cláusula mergeable (uso inseguro)

```
#include <stdio.h>
void foo () { /* tsk_mergeable.c */
int x = 2;
    #pragma omp task mergeable firstprivate(x)
        x++;
    #pragma omp taskwait
    printf("%d\n",x); // imprime 2 ou 3
}
```

Cláusula untied

```
#pragma omp task untied  
{structured-block}
```

- As tarefas são vinculadas por padrão (quando nenhuma cláusula **untied** está presente)
 - Tarefas vinculadas são sempre executadas pela mesma *thread* (não necessariamente a que a criou)
 - Tarefas vinculadas podem apresentar problemas de desempenho
- Os programadores podem especificar que as tarefas sejam desvinculadas (agendar com flexibilidade)
 - Podem potencialmente ser executadas por qualquer *thread* (do time)
 - Isso dá ao sistema de execução mais flexibilidade para agendar as tarefas
 - Não se harmoniza bem com recursos baseados em *threads*: *thread-id*, *threadprivate*, *critical regions*...

Cláusula priority

#pragma omp task priority (N)

- A cláusula **priority** é uma sugestão para o escalonador de prioridade da tarefa gerada, através de um valor numérico não negativo.
- As tarefas com valor numérico mais alto têm recomendação para serem executadas antes das tarefas de prioridade mais baixa.
- A prioridade padrão, quando nenhuma cláusula **priority** é especificada, é zero (a menor prioridade).
- Se um valor for especificado na cláusula **priority** for maior do que o valor máximo definido (ICV **max-task-priority-var**), então o ambiente de execução usará o valor dessa ICV.
- Um programa que dependa da ordem de execução das tarefas determinada por esse valor de prioridade pode ter um comportamento indeterminado, já que a prioridade é apenas uma sugestão e não é obrigatória.

Exemplo Fibonacci

- Programa para cálculo dos números de Fibonacci
- Os números de Fibonacci são números na seguintes sequência inteira.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,

- Em termos matemáticos, a sequência F_n dos números de Fibonacci é definida pela seguinte relação de recorrência:

$$F_n = F_{n-1} + F_{n-2}$$

- Com os seguintes valores iniciais

$$F_0 = 0 \text{ e } F_1 = 1.$$

Exemplo Fibonacci

```
#include <stdio.h>
long fib(int n)
{
    if (n < 2)
        return n;
    int x = fib(n - 1);
    int y = fib(n - 2);
    return x + y ;
}

int main(void)
{
    int n = 40;
    fib(n);
    return 0;
}
```

Escopo das variáveis

```
int fib ( int n ) {  
    int x,y;  
    if ( n < 2 ) return n;  
    #pragma omp task  
    x = fib(n-1);  
    #pragma omp task  
    y = fib(n-2);  
    #pragma omp taskwait  
  
    return x+y;  
}
```

n é privado em ambas tarefas

As variáveis privadas de um tarefa são indefinidas fora da tarefa.

Escopo das variáveis

```
int fib (int n ) {  
    int x,y;  
    if ( n < 2 ) return n;  
    #pragma omp task shared (x)  
    x = fib(n-1);  
    #pragma omp task shared (y)  
    y = fib(n-2);  
    #pragma omp taskwait  
    return x+y  
}
```

n é privado em ambas tarefas

*Declarar x e y compartilhados
é uma boa solução*

Solução correta do ponto de vista funcional,
porém com baixo desempenho, porque a
granularidade das tarefas é muito pequena. O
desempenho pode ser melhorado utilizando
estratégias de corte.

Exemplo Fibonacci

```
#include <stdio.h>
long fib(int n)
{
    int x, y;
    if (n < 2)
        return n;
    #pragma omp task shared (x)
        x = fib(n - 1);
    #pragma omp task shared (y)
        y = fib(n - 2);
    #pragma omp taskwait
        return x + y ;
}
```

```
int main(void)
{
    int n = 40;
    #pragma omp parallel
    #pragma omp single
        fib(n);
    return 0;
}
```

Escopo das Variáveis

- Escopo de dados das cláusulas:
`shared(list)`, `private(list)`, `firstprivate(list)`
- Variáveis estáticas e globais são compartilhadas.
- Variáveis de armazenamento automático (locais) são privadas.
- Variáveis de tarefas órfãs são `firstprivate` por padrão.
- Variáveis de tarefas com pais herdam os atributos de compartilhamento da tarefa pai.
- Variáveis são `firstprivate` a menos que compartilhadas no contexto em que estão colocadas.

Escopo das Variáveis

Variáveis com duração de armazenamento automática declaradas dentro da construção task são privadas.

```
#pragma omp task
{
int x = MN;
// Escopo de x: private
}
```

Variáveis com duração de armazenamento estática declaradas dentro da construção task são compartilhadas.

```
#pragma omp task
{
static int y
// Escopo de y: shared
}
```

```
int *p;
p = malloc(sizeof(float)*SIZE);
#pragma omp task
{
// *p: shared
}
```

Objetos de armazenamento automático são compartilhados (malloc, new,...)

Escopo das Variáveis

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4;
        #pragma omp task
        {
            int e = 5;
            // Escopo de a:
            // Escopo de b:
            // Escopo de c:
            // Escopo de d:
            // Escopo de e:
        }
    }
}
```

Escopo das Variáveis

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4;
        #pragma omp task
        {
            int e = 5;
            // Escopo de a: shared
            // Escopo de b: private
            // Escopo de c: shared
            // Escopo de d: firstprivate
            // Escopo de e: private
        }
    }
}
```

Escopo das Variáveis – na dúvida use default (none)

```
int a = 1;
void foo() {
    int b = 2, c = 3;
    #pragma omp parallel private(b)
    {
        int d = 4;
        #pragma omp task
        {
            int e = 5;
            // Escopo de a: shared valor de a: 1
            // Escopo de b: private valor de b: 0/indefinido
            // Escopo de c: shared valor de c: 3
            // Escopo de d: firstprivate valor de d: 4
            // Escopo de e: private valor de e: 5
        }
    }
}
```

Exemplo Quicksort

- Um algoritmo frequentemente utilizado para ordenação é o quicksort.
- Utiliza uma estratégia de dividir para conquistar com os seguintes passos:
 - Dividir o vetor com um pivô, de tal modo que:
 - Todos os elementos à esquerda são menores
 - Todos os elementos à direita são maiores ou iguais
 - Repetir o procedimento para a parte direita e esquerda até completar

Exemplo Quicksort

- Um algoritmo recursivo sequencial

```
1 void Quicksort(int64_t *a, int64_t lo, int64_t hi)
2 {
3     if ( lo < hi ) {
4         int64_t p = partitionArray(a, lo, hi);
5
6         (void) Quicksort(a, lo, p - 1); // Left branch
7
8         (void) Quicksort(a, p + 1, hi); // Right branch
9     }
10 }
```


Exemplo Quicksort

```
1 #pragma omp parallel default(none) shared(a,nelements)
2 {
3     #pragma omp single nowait
4     { (void) Quicksort(a, 0, nelements-1); }
5 } // End of parallel region
```

```
1 void Quicksort(int64_t *a, int64_t lo, int64_t hi)
2 {
3     if ( lo < hi ) {
4         int64_t p = partitionArray(a, lo, hi);
5
6         #pragma omp task default(none) firstprivate(a,lo,p)
7         {(void) Quicksort(a, lo, p - 1);} // Left branch
8
9         #pragma omp task default(none) firstprivate(a,hi,p)
10        {(void) Quicksort(a, p + 1, hi);} // Right branch
11    }
12 }
```

Exemplo Quicksort

- Quando o tamanho do vetor for muito pequeno, é melhor utilizar o algoritmo sequencial.
- Isso pode ser realizado com a cláusula `if`.
- O modelo tradicional de OpenMP *threads* pode ser considerado desatualizado
- O uso de `tasks` é mais flexível e alivia o programador de pensar no mapeamento da execução da `tasks` para as *threads*.

Laboratório 1



<https://bit.ly/erad-se-lab1>

Segunda Parte

Exemplo Lista Encadeada

```
ponteiro = inicio_lista;  
.....  
while(ponteiro) {  
    (void) faz_trabalho_independente (ponteiro);  
    ponteiro = ponteiro->prox ;  
} // Fim do laço while  
  
.....
```

Difícil de realizar sem utilizar tasks: primeiro contar o número de iterações e depois converter o while para um laço for.

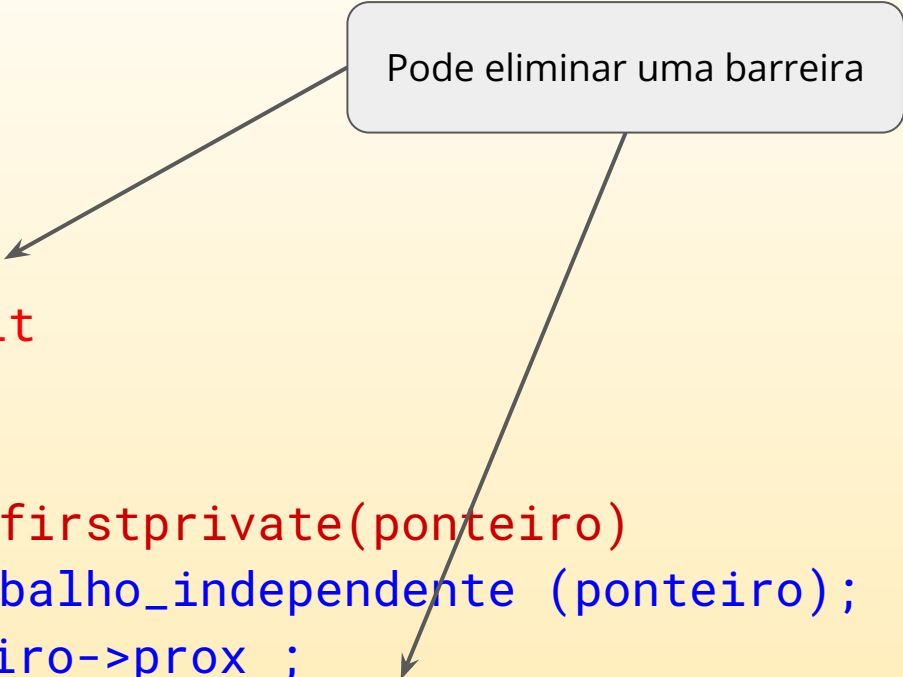
Exemplo Lista Encadeada

```
ponteiro = inicio_lista;
#pragma omp parallel
{
    #pragma omp single
    {
        while(ponteiro) {
            #pragma omp task firstprivate(ponteiro)
            (void) faz_trabalho_independente (ponteiro);
            ponteiro = ponteiro->prox ;
        }
    } // Fim do single
} // Fim da região paralela
```

A tarefa OpenMP Task é especificada aqui (executadas em paralelo)

Exemplo Lista Encadeada

```
ponteiro = inicio_lista;
#pragma omp parallel
{
    #pragma omp single nowait
    {
        while(ponteiro) {
            #pragma omp task firstprivate(ponteiro)
            (void) faz_trabalho_independente (ponteiro);
            ponteiro = ponteiro->prox ;
        }
    } // Fim do single - sem barreira implícita (nowait)
} // Fim da região paralela - barreira implícita
```



Pode eliminar uma barreira

Dependência de Tarefas

- Em OpenMP, tarefas podem depender umas das outras com base nos dados que elas acessam.
- As dependências entre tarefas são especificadas usando a cláusula `depend` da diretiva `task`. A cláusula `depend` pode ser usada para especificar dependências `in`, `out` e `inout`.
- Uma dependência `in` significa que a tarefa precisa dos dados produzidos por outra tarefa para ser executada.
- Uma dependência `out` significa que a tarefa produz dados que serão usados por outra tarefa.
- Uma dependência `inout` significa que a tarefa produz dados que serão usados por outra tarefa e que serão modificados pela outra tarefa.

Dependência de Tarefas

```
#pragma omp task  
  x = f();  
#pragma omp task  
  y = g(x);
```

```
#pragma omp task depend(out:x)  
  x = f();  
#pragma omp task depend(in:x)  
  y = g(x);
```

Dependência de Tarefas

```
#include <stdio.h>
int main() {
    int x;
    #pragma omp parallel
    #pragma omp single nowait
```

A primeira tarefa vai escrever na variável "x",
a segunda tarefa vai ler e escrever na/da variável "x", e
a terceira tarefa vai ler da variável "x".

```
{
    #pragma omp task depend(out: x)
        x = 1;
    #pragma omp task depend(inout: x)
    {
        printf("Eu acho que o número é %d\n", x);
        x++;
    }
    #pragma omp task depend(in: x)
        printf("Eu acho que o número final é %d\n", x);
}
return 0;
}
```

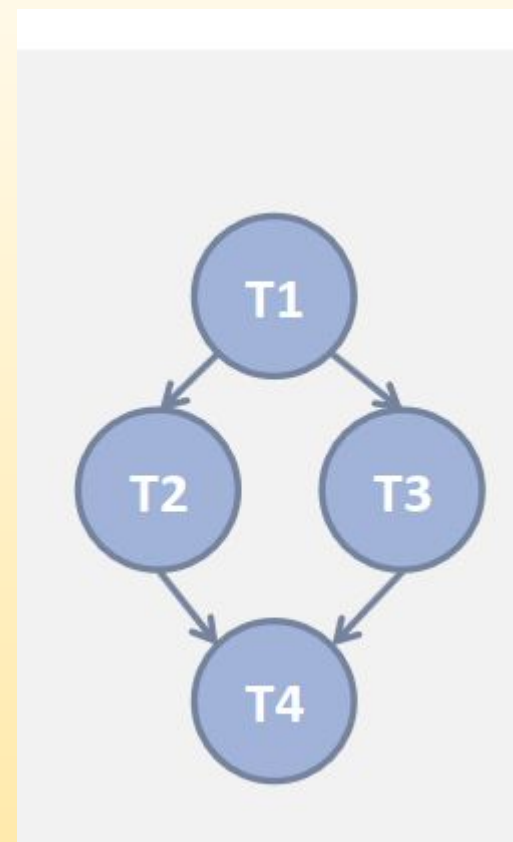
Dependência de Tarefas

```
int x = 0;
#pragma omp parallel
#pragma omp single
{
  #pragma omp task depend(inout: x) //T1
  { ... }

  #pragma omp task depend(in: x) //T2
  { ... }

  #pragma omp task depend(in: x) //T3
  { ... }

  #pragma omp task depend(inout: x) //T4
  { ... }
}
```



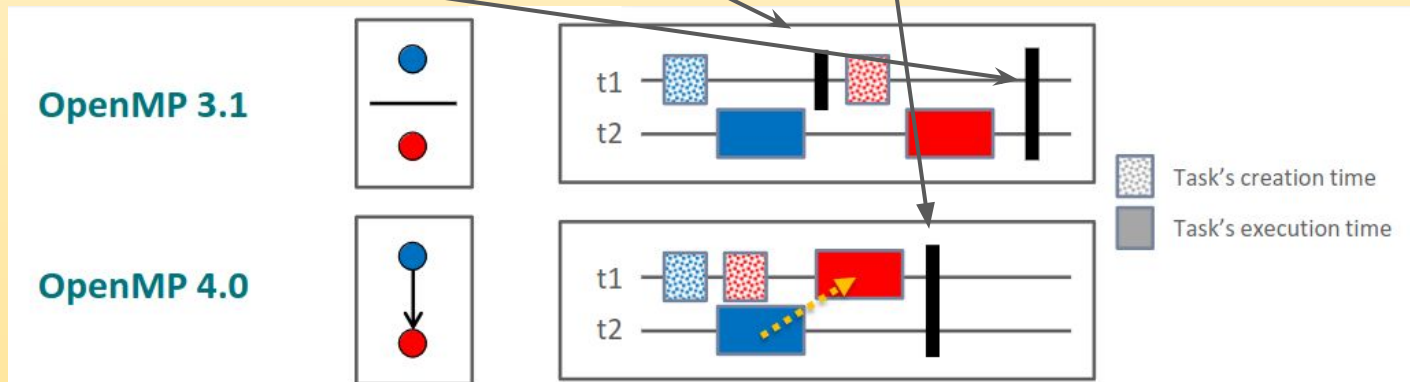
Dependência de Tarefas

OpenMP 3.1

```
int x = 0;
#pragma omp parallel
#pragma omp single
{
    #pragma omp task
    printf("%d\n", x);
    #pragma omp taskwait
    #pragma omp task
    x++;
}
```

OpenMP 4.5

```
int x = 0;
#pragma omp parallel
#pragma omp single
{
    #pragma omp task depend(in: x)
    printf("%d\n", x);
    #pragma omp task depend(inout: x)
    x++;
}
```



Diretivas barrier e taskwait

- Diretiva barrier
 - Todas as tarefas criadas por qualquer *thread* do time atual garantidamente são completadas na saída da diretiva **barrier**
 - E em todas as outras barreiras implícitas em **parallel**, **sections**, **for**, **single**, etc.

pragma omp barrier

- Diretiva taskwait
 - A tarefa que encontra esta diretiva é suspensa até que as tarefas filhas sejam completadas.
 - Aplica-se apenas aos filhos diretos, não aos descendentes!
 - Inclui um ponto implícito de agendamento de tarefas (TSP).

pragma omp taskwait

Exemplo Cholesky

- Decomposição de Cholesky: fatora uma matriz simétrica e definida positiva **A** na forma

$$\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$$

- em que **L** é uma matriz triangular inferior.
- Essa decomposição é utilizada para resolver sistemas lineares de forma eficiente, além de aplicações em otimização, filtros de Kalman e métodos de Monte Carlo para geração de distribuições gaussianas multivariadas.
- Padrões de sincronização complexos podem ser resolvidos dividindo fases computacionais e usando construções como `taskwait` ou `taskgroup`.
- Isso, no entanto, exige uma complexa análise de código.
- A melhoria da programabilidade pode ser alcançada ao considerar as dependências do OpenMP, o que também aprimora a composição de código.

Exemplo Cholesky

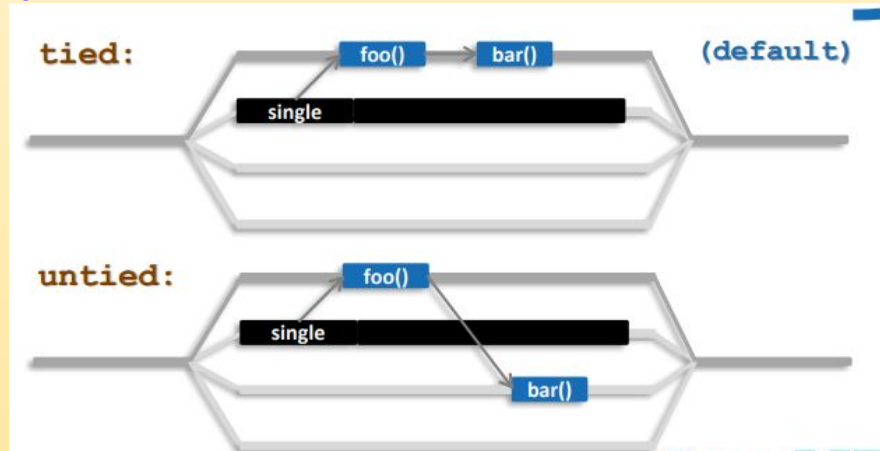
```
void cholesky(int ts, int nt, double* a[nt][nt]) {  
    // Percorre cada bloco diagonal da matriz  
    for (int k = 0; k < nt; k++) {  
        potrf(a[k][k], ts, ts); // Fatora o bloco diagonal atual  
        // Resolve os blocos na mesma "linha"  
        // usando o fator triangular recém-calculado  
        for (int i = k + 1; i < nt; i++) {  
            #pragma omp task  
            trsm(a[k][k], a[k][i], ts, ts);  
        }  
        // Espera todas as tarefas de trsm terminarem  
        #pragma omp taskwait  
        // Atualiza os blocos abaixo/à direita do bloco k  
        for (int i = k + 1; i < nt; i++) {  
            for (int j = k + 1; j < i; j++) {  
                #pragma omp task  
                dgemm(a[k][i], a[k][j], a[j][i], ts, ts);  
            }  
            #pragma omp task  
            syrkm(a[k][i], a[i][i], ts, ts);  
        }  
        // Espera todas as atualizações terminarem  
        // antes de passar para o próximo bloco diagonal  
        #pragma omp taskwait  
    }  
}
```

Diretivas taskyield

- Pontos de agendamento de tarefas (e na diretiva `taskyield`)
 - → Tarefas podem ser suspensas/reiniciadas nos Pontos de Agendamento de Tarefas (TSPs) → com algumas restrições adicionais para evitar problemas de bloqueio
 - Pontos de agendamento implícitos (criação, sincronização, ...)
 - Ponto de agendamento explícito: a diretiva `taskyield`

`#pragma omp taskyield`

```
#pragma omp parallel
#pragma omp single {
    #pragma omp task {
        foo();
    }
    #pragma omp taskyield
    bar()
}
```



Diretiva taskgroup

- Diretiva taskgroup

`#pragma omp taskgroup [cláusula]`

- Especifica a espera pelo término das tarefas filhas e dos seus descendentes.
- Sincronização mais profunda que `taskwait`
- Pode ser restrita a um subconjunto de tarefas (ao contrário da barreira)
- Pode ser usada para o cancelamento da tarefa
- onde cláusula pode ser: `task_reduction(reduction-identifier: list-items)` ou `allocate`

Diretivas taskwait vs taskgroup

center@#01%101

Taskwait

```
int main()
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp task
        {
            #pragma omp critical
            printf ("Task_1\n");

            #pragma omp task
            {
                sleep(1);
                #pragma omp critical
                printf ("Task_2\n");
            }

            #pragma omp taskwait

            #pragma omp task
            {
                #pragma omp critical
                printf ("Task_3\n");
            }
        }
    }
}
```

Taskgroup

```
int main()
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp taskgroup
        {
            #pragma omp task
            {
                #pragma omp critical
                printf ("Task_1\n");

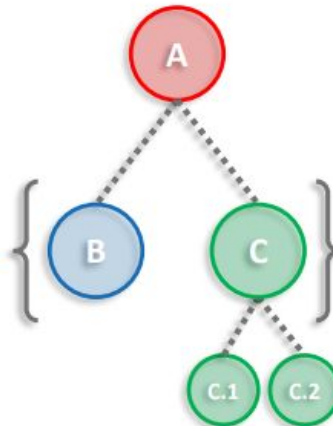
                #pragma omp task
                {
                    sleep(1);
                    #pragma omp critical
                    printf ("Task_2\n");
                }
            }
        } /* end of taskgroup */

        #pragma omp task
        {
            #pragma omp critical
            printf ("Task_3\n");
        }
    }
}
```

Diretivas taskwait vs taskgroup

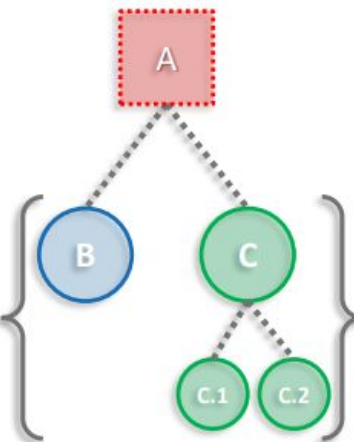
```
#pragma omp parallel
#pragma omp single
{
    #pragma omp task :A
    {
        #pragma omp task :B
        { ... }
        #pragma omp task :C
        { ... #C.1; #C.2; ... }
        #pragma omp taskwait
    }
} // implicit barrier will wait for C.x
```

wait for...



```
#pragma omp parallel
#pragma omp single
{
    #pragma omp taskgroup :A
    {
        #pragma omp task :B
        { ... }
        #pragma omp task :C
        { ... #C.1; #C.2; ... }
    } // end of taskgroup
}
```

wait for...



Diretiva taskloop

- Paraleliza um laço utilizando OpenMP tasks
- Divide o laço em partes
- Cria uma task para cada parte do laço
- Sintaxe (C/C++)

```
#pragma omp taskloop [simd] [clausula[,] clausula],...]  
for-loops
```

Diretiva taskloop

- A diretiva taskloop pode utilizar as seguintes cláusulas:
 - `shared, private`
 - `firstprivate, lastprivate`
 - `default (shared, none)`
 - `collapse`
 - `final, untied, mergeable`
 - `grainsize(grain-size)`
 - Os pedaços têm entre `grain-size` e $2 * \text{grain-size}$ iterações de laço
 - `num_tasks(num-tasks)`
 - Cria `num-tasks` tarefas para executar as iterações do laço
 - Se nenhuma das cláusulas anteriores estiver presente, o número de blocos e o número de iterações por bloco são definidos pela implementação.

Diretiva taskloop

- Paraleliza um laço criando tarefas para uma ou mais iterações do laço.
- Divide o laço em blocos (*chunks*) e cria uma tarefa para cada bloco de iterações.
- Herda cláusulas tanto das diretivas de compartilhamento de trabalho (*worksharing*) quanto da diretiva *task*.
- Em alguns casos, proporciona melhor balanceamento de carga.
- Cláusulas adicionais:
- *grainsize*(*grain-size*): os blocos possuem pelo menos *grain-size* e no máximo $2 \times \text{grain-size}$ iterações do laço.
- *num_tasks*(*num-tasks*): cria *num-tasks* tarefas para executar as iterações do laço.

Diretiva taskloop

```
#pragma omp parallel shared(a, b, c)
{
    #pragma omp single
    {
        #pragma omp task
        // pode executar concorrentemente
        long_running_comp();
        // pode executar concorrentemente
        #pragma omp taskloop grainsize(1000)
        for (int i = 0; i < 1000000; ++i) {
            c[i] = a[i] + b[i];
        }
    }
}
```

Diretiva taskloop simd

- Divide um laço em blocos, cria uma tarefa para cada bloco do laço.
- Cada tarefa gerada aplicará (internamente) SIMD a cada bloco do loop.

```
#pragma omp taskloop simd [clausula[,] clausula]...  
{structured-for-loops}
```


Diretiva taskloop simd

- Admite quaisquer das cláusulas permitidas pelas diretivas `taskloop` ou `simd`. Exemplos:
 - `safelen(length)`: Determina o número máximo de iterações que podem executar concorrentemente via SIMD sem dependências de dados.
 - `simdlen(length)`: Solicita ao hardware um comprimento específico e preferencial do vetor SIMD.
 - `aligned(list:alignment)`: Informa ao compilador que os ponteiros de alocação de memória estão alinhados a limites específicos de bytes, para otimizar a carga SIMD.

Diretiva cancel

```
#pragma omp cancel [parallel][sections][for][taskgroup]
```

- A diretiva `cancel` ativa o cancelamento da região mais interna envolvida do tipo especificado.
- A diretiva `omp cancel` é uma diretiva independente (*stand-alone directive*). O conjunto de *threads* associado à região de cancelamento é a equipe (*team*) atual, e a região à qual o cancelamento se aplica é a região mais interna envolvida pela diretiva e cujo tipo é especificado na cláusula utilizada.
- O mecanismo de cancelamento é habilitado somente quando a variável de ambiente `OMP_CANCELLATION` está definida como `true`. Quando o mecanismo de cancelamento está habilitado, a diretiva `omp cancel` ativa o cancelamento da região associada; caso contrário, a diretiva é simplesmente ignorada.

Diretiva cancellation point

- O usuário pode solicitar o cancelamento de uma construção (*construct*).
- As *threads* ou tarefas serão canceladas e a execução continuará após o término da construção.
- Aplicável às construções: *parallel*, *for*, *taskgroup* e *sections*.
- As *threads* ou tarefas interrompem sua execução em um determinado ponto de cancelamento, e não imediatamente.

Diretiva cancellation point

```
#pragma omp parallel shared(matrix)
{
    #pragma omp for
    for (int row = 0; row < rows; row++)
    {
        for (int col = 0; col < cols; col++)
        {
            if (matrix(row, col) == 0)
            {
                #pragma omp cancel for
            }
        }
        #pragma omp cancellation point for
    }
}
```

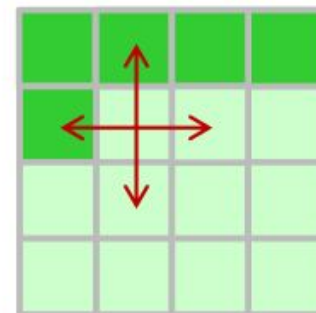
Estudo de Caso (Gauss-Seidel)

```
void serial_gauss_seidel(int tsteps, int size,  
                        double p[size][size]) {  
    for (int t = 0; t < tsteps; ++t) {  
        for (int i = 1; i < size - 1; ++i) {  
            for (int j = 1; j < size - 1; ++j) {  
                p[i][j] = 0.25 *  
                    (p[i][j-1] + // oeste  
                     p[i][j+1] + // leste  
                     p[i-1][j] + // norte  
                     p[i+1][j]); // sul  
            }  
        }  
    }  
}
```

Cada célula depende de:

- duas células (norte e oeste) que são computadas no passo atual, e
- duas células (sul e leste) que foram computadas no passo anterior.

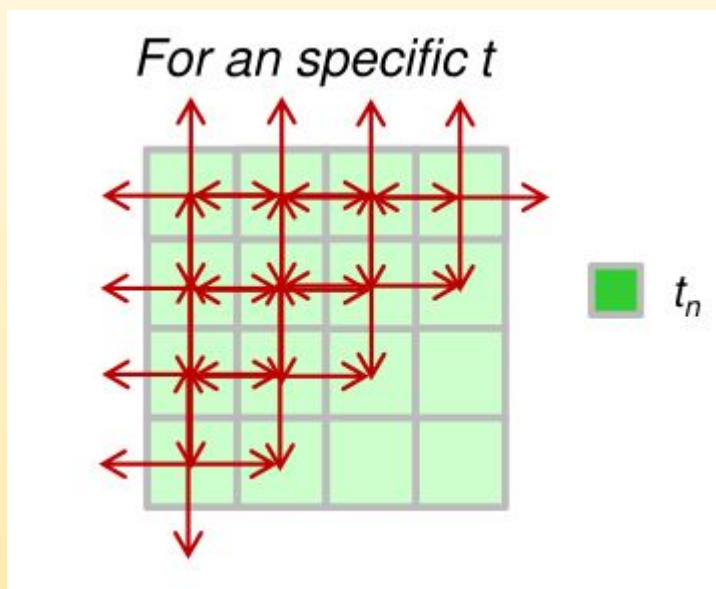
For a specific t , i and j



t_n

Estudo de Caso (Gauss-Seidel)

- Podemos explorar a frente de onda para obter paralelismo!

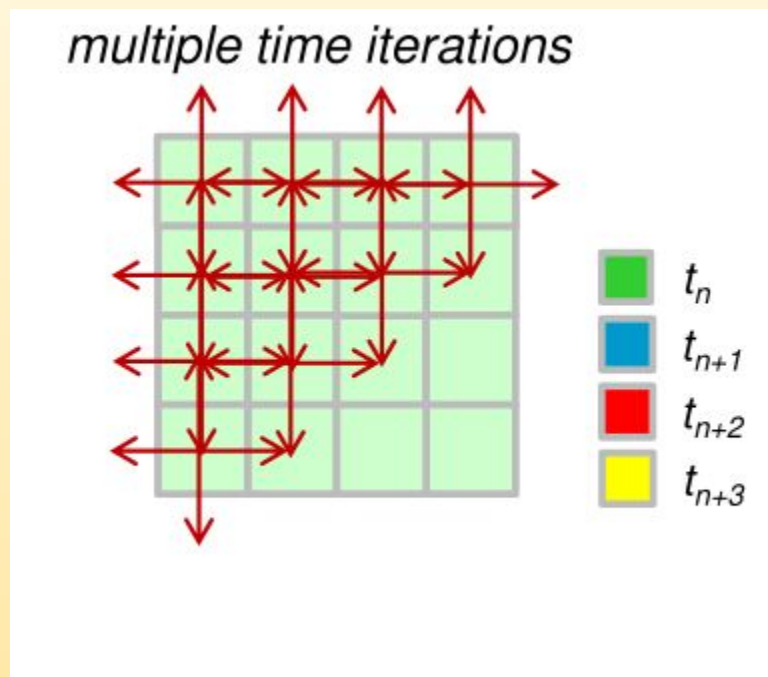


Estudo de Caso (Gauss-Seidel)

```
void gauss_seidel(int tsteps, int size, int TS, double (*p)[size]) {
    int NB = size / TS;
    #pragma omp parallel
    for (int t = 0; t < tsteps; ++t) {
        for (int diag = 0; diag < NB; ++diag) { // Primeiras NB diagonais
            #pragma omp for
                for (int d = 0; d <= diag; ++d) {
                    int ii = d;
                    int jj = diag - d;
                    for (int i = 1+ii*TS; i < ((ii+1)*TS); ++i)
                        for (int j = 1+jj*TS; j < ((jj+1)*TS); ++j)
                            p[i][j] = 0.25 * (p[i][j-1] + p[i][j+1] +
                                p[i-1][j] + p[i+1][j]);
                }
        }
        for (int diag = NB-1; diag >= 0; --diag) { // Últimas NB diagonais
            // Código similar ao laço anterior
        }
    }
}
```

Estudo de Caso (Gauss-Seidel)

- Podemos explorar a frente de onda em vários intervalos de tempo para obter MAIS paralelismo!



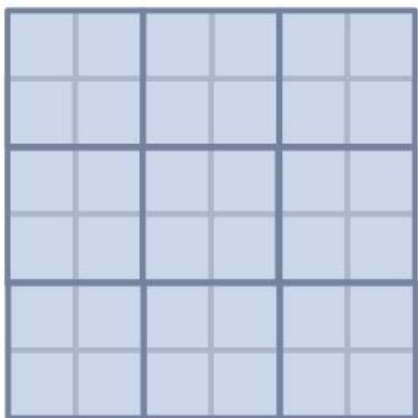
Estudo de Caso (Gauss-Seidel)

```
#define MIN(a,b) ((a) < (b) ? (a) : (b))
void gauss_seidel(int tsteps, int size, int TS, double p[size][size])
{
    int NB = (size-2+TS-1)/TS;
    char dep[NB+2][NB+2];
    #pragma omp parallel
    #pragma omp single
    for (int t=0; t<tsteps; t++)
        for (int I=1; I<=NB; I++)
            for (int J=1; J<=NB; J++) {
                int ii=1+(I-1)*TS, ie=MIN(ii+TS,size-1);
                int jj=1+(J-1)*TS, je=MIN(jj+TS,size-1);
                #pragma omp task firstprivate(ii,ie,jj,je) \
                    depend(inout: dep[I][J]) \
                    depend(in: dep[I-1][J],dep[I+1][J], \
                        dep[I][J-1],dep[I][J+1])
                for (int i=ii; i<ie; i++)
                    for (int j=jj; j<je; j++)
                        p[i][j]=0.25*(p[i][j-1]+p[i][j+1]+
                            p[i-1][j]+p[i+1][j]);
            }
}
```

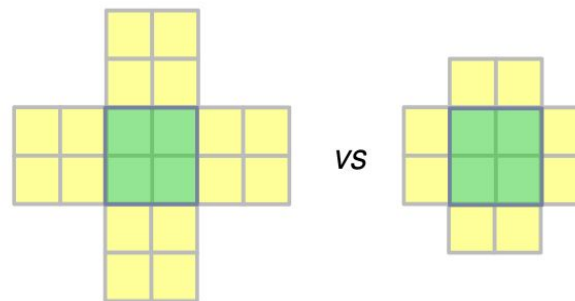
Estudo de Caso (Gauss-Seidel)

- Os valores de **dep** não precisam ser inicializados, pois seu conteúdo nunca é lido: apenas os seus endereços são usados como identificadores de dependência.
- A barreira implícita ao final da região **parallel** garante que todas as tarefas terminem antes de a função retornar.

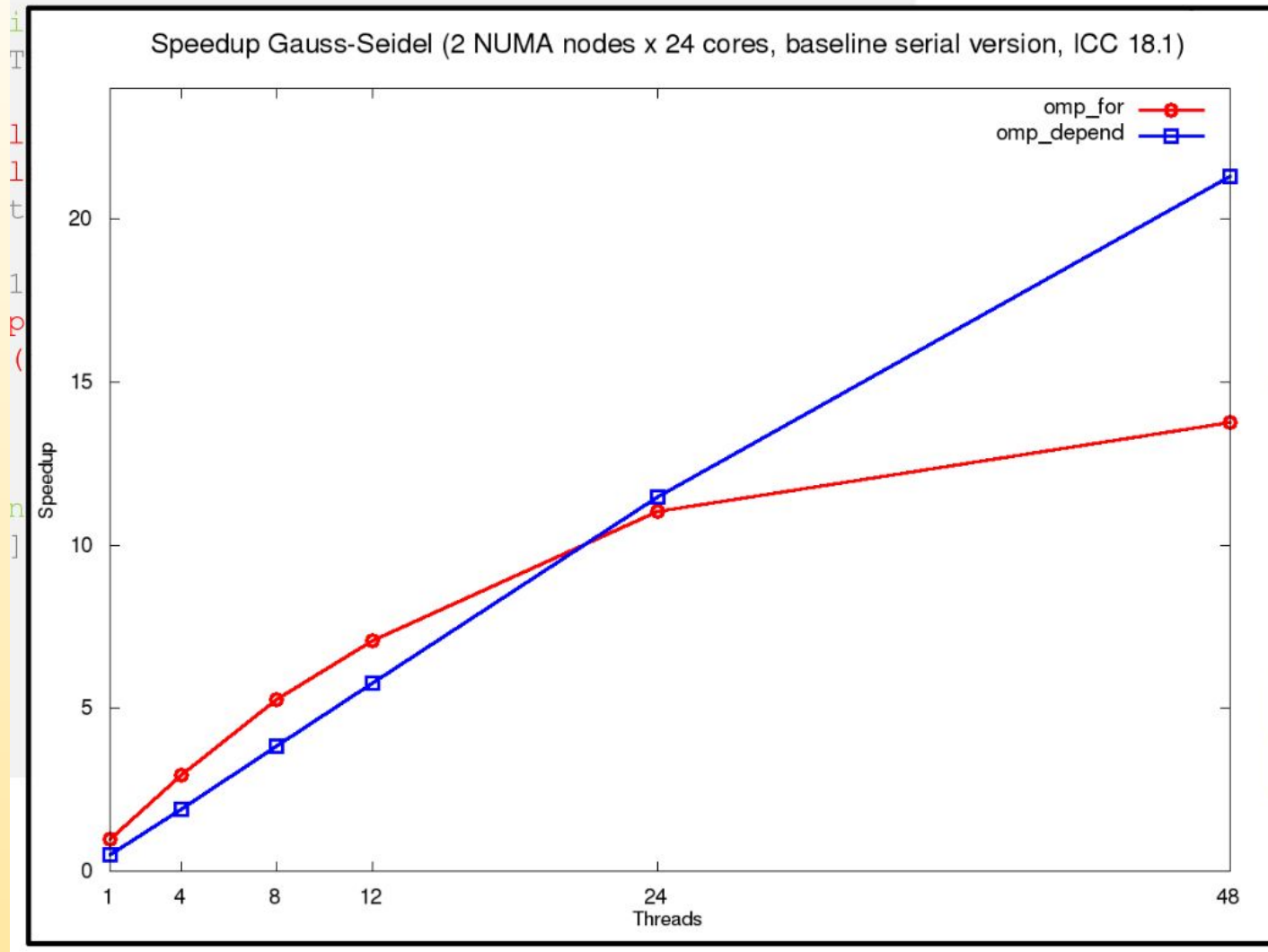
inner matrix region



Q: Why do the input dependences depend on the whole block rather than just a column/row?



Estudo de Caso (Gauss-Seidel)



Laboratório 2



<https://bit.ly/erad-se-lab2>

Referências

<https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>

<https://www.openmp.org/wp-content/uploads/OpenMP-UMT-Tasking-1.pdf>

<https://openmp.org/wp-content/uploads/sc13.tasking.ruud.pdf>

https://hpc-wiki.info/hpc/OpenMP_in_Small_Bites/Tasking_and_Data_Scoping

https://docs.oracle.com/cd/E60778_01/html/E60751/gozsl.html

https://blog.rwth-aachen.de/itc-events/files/2021/02/11-openmp-CT-tasking_dependencies.pdf

<https://hpc2n.github.io/Task-based-parallelism/branch/master/task-basics-1/>